

MLOps Heart Disease Risk Classification

End-to-end model development, CI/CD, deployment - monitoring

Assignment 1

Name: Amruthavalli Kuncha

Student ID: 2024ac05402

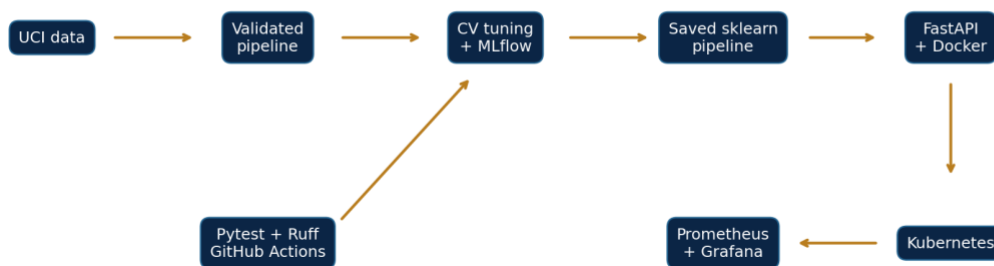
GitHub repository: <https://github.com/amruthavallikuncha/mlops>

Summary

MLOps Heart Disease Risk prediction project using the official UCI Cleveland heart-disease data into a reproducible binary classifier and as an API to predict using new data for the end users. Two models were tuned with five-fold stratified cross-validation. Logistic regression was selected by the criteria highest training-partition CV ROC-AUC (0.91) and reached 0.93 ROC-AUC on the untouched test set.

MLflow records experiments, FastAPI serves predictions, Pytest and Ruff checks code quality, Docker and Kubernetes package deployment, Prometheus and Grafana expose operational signals.

System architecture



A framework from data acquisition to monitored deployment.

Verified outcome

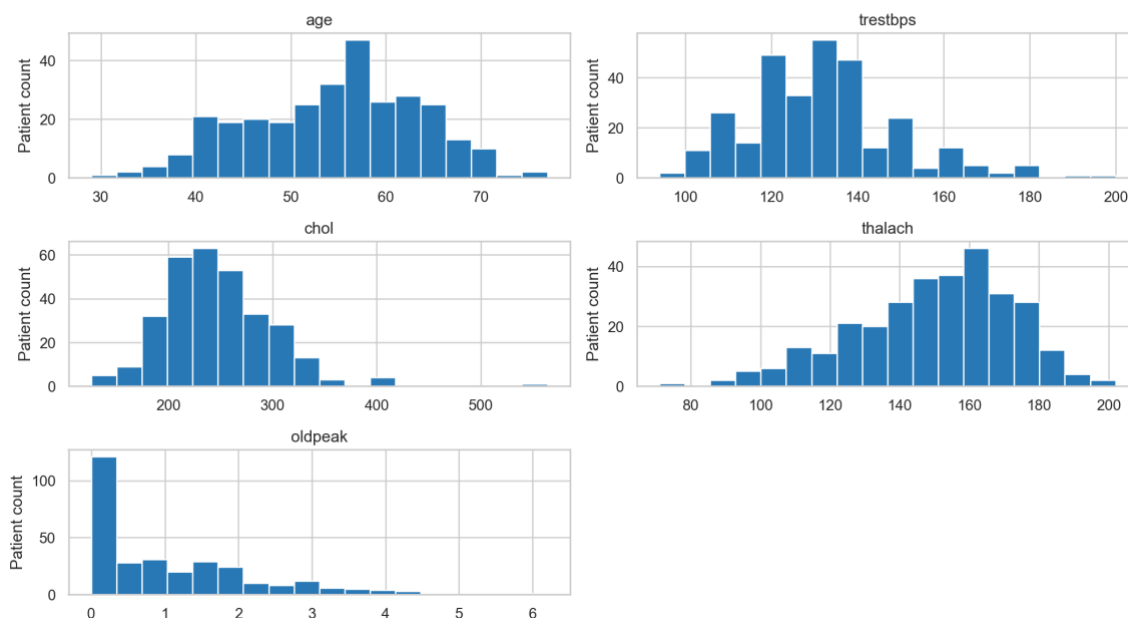
- 303 official UCI records, 13 predictors, 45.87% positive outcome.
- Six tests passing with 92% line coverage, lint checks passing.
- Selected model: logistic regression, C=0.2, no class weighting.
- Saved artifact includes imputation, encoding, scaling and classifier.

1. Data acquisition and exploratory data analysis

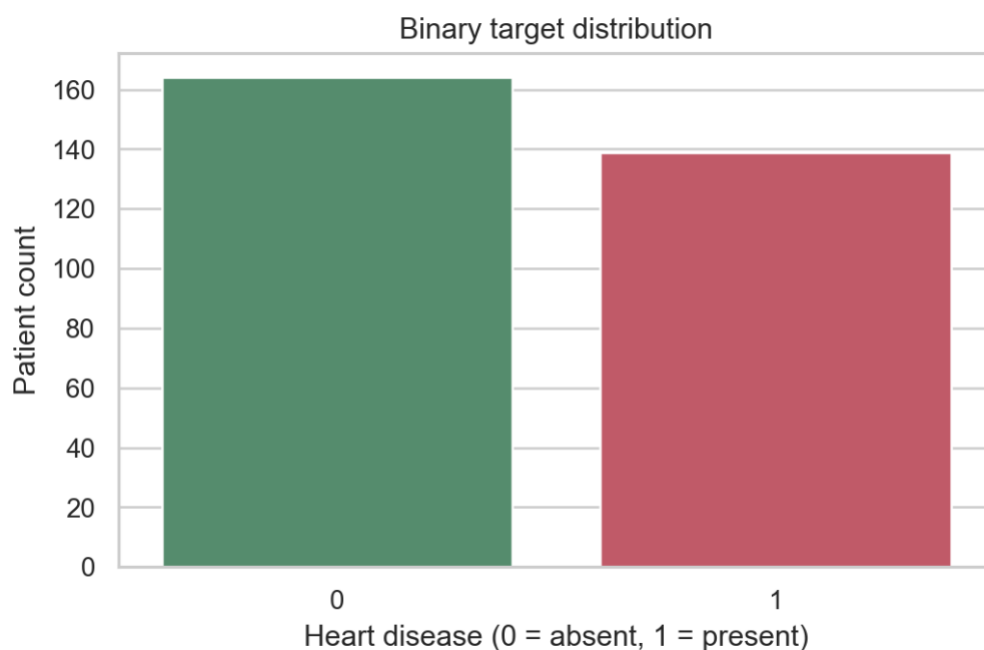
The processed Cleveland cohort is downloaded directly from the UCI Machine Learning Repository. The raw file digest sha 256 one provides an auditable data version. Original severity values 1–4 are mapped to presence and 0 to absence.

The cleaning audit found no duplicate records, four missing values in ca and two in thal. Missing predictors are retained for leakage-safe pipeline imputation; the outcome has no missing values.

Distribution of continuous clinical measurements



Continuous-feature distributions across 303 records.

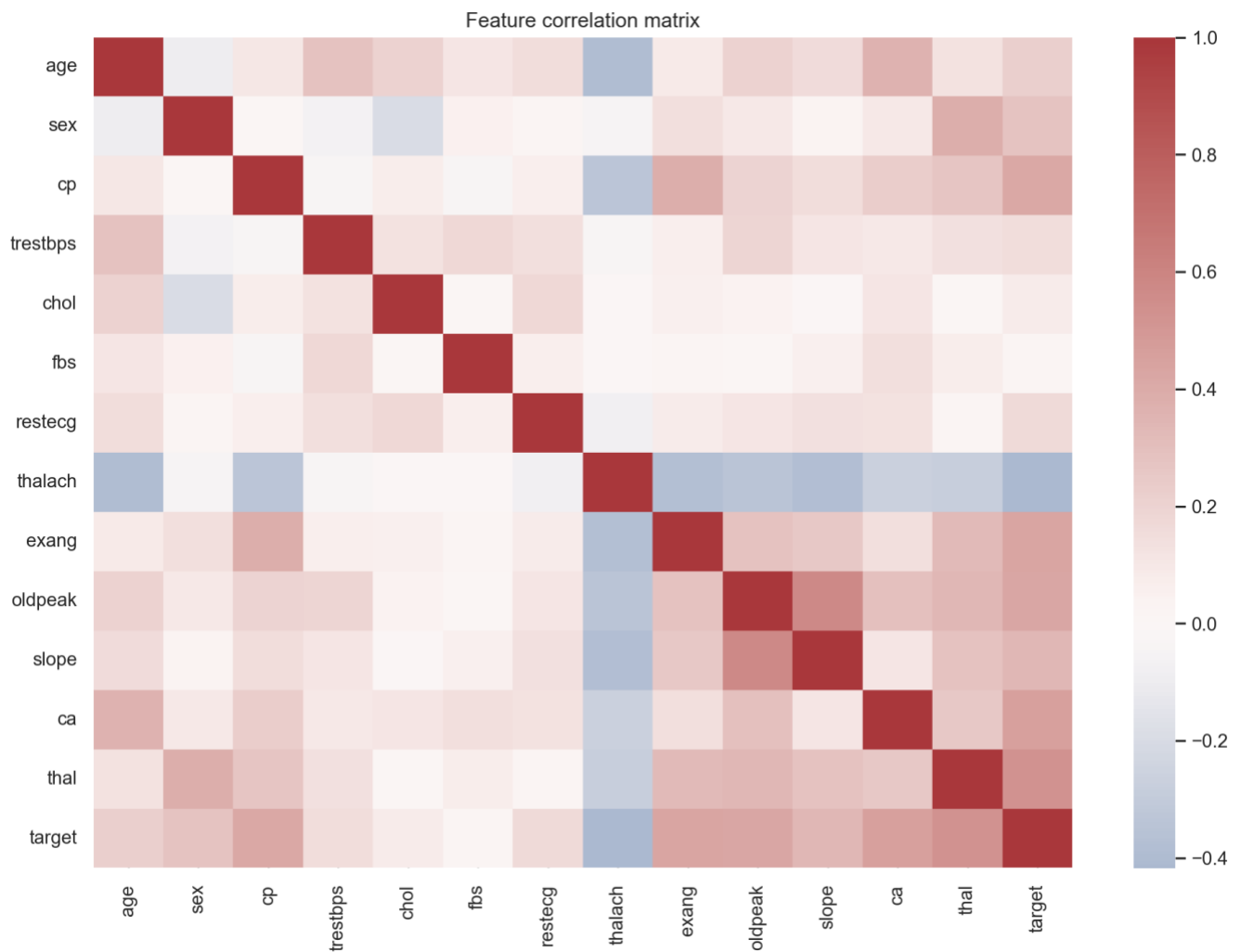


Binary outcome distribution (positive rate 45.87%).

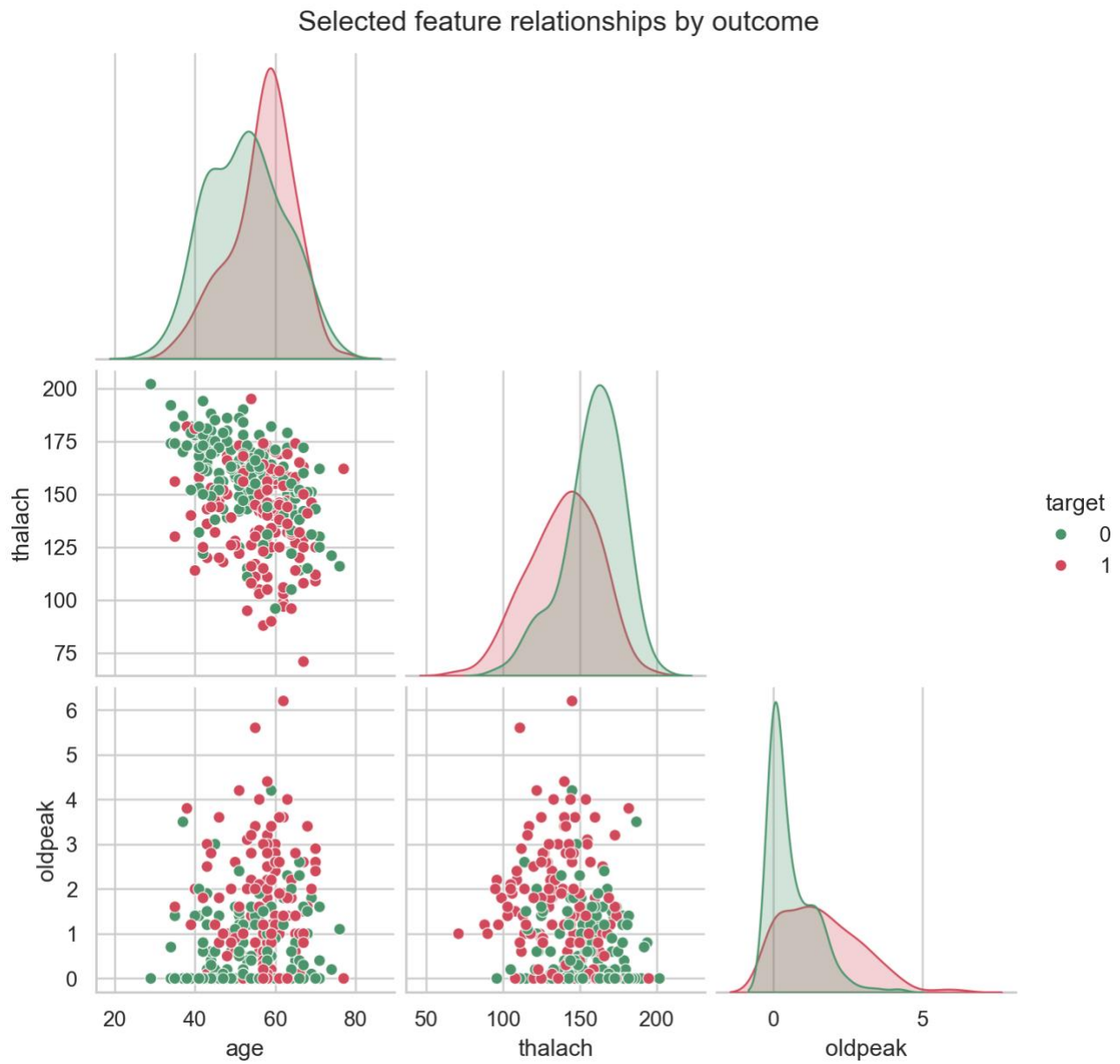
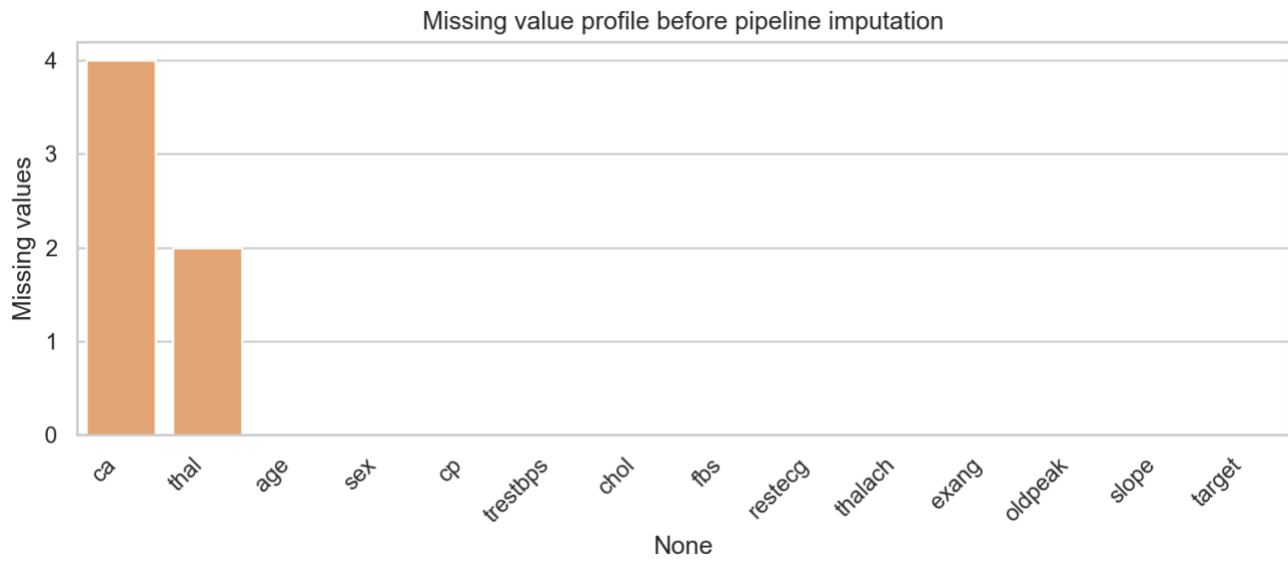
2. Feature engineering and evaluation design

Numeric features: age, resting blood pressure, cholesterol, maximum heart rate and ST depression use median imputation with missingness indicators and standardization. Eight coded features use most frequent imputation and one hot encoding with unknown category tolerance.

An 80/20 stratified split with seed produces 242 training and 61 held out records. Grid search operates only on the training partition using five shuffled stratified folds and ROC - AUC scoring to keep test data outside preprocessing and selection.



Numeric correlation matrix.

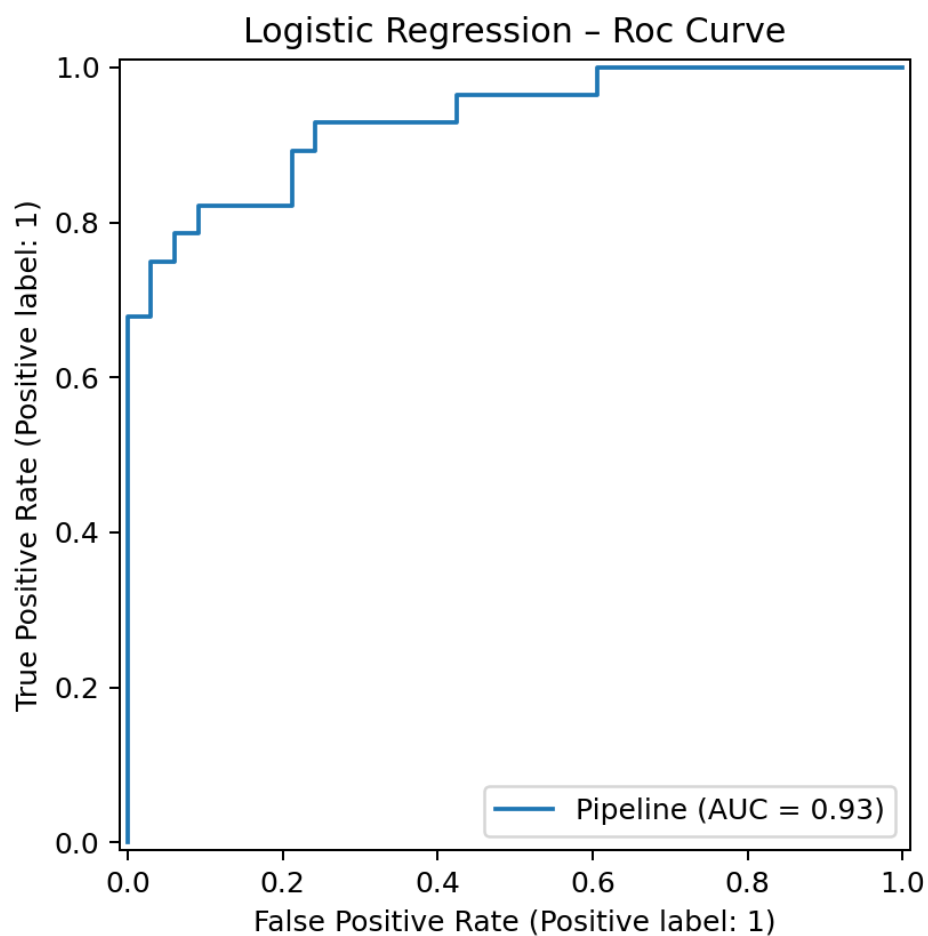


Model development and selection

Logistic regression explores four regularization strengths and optional balanced weights. Random forest explores tree count, depth, minimum leaf size and optional balanced weights. The selection rule is highest cross-validated ROC - AUC, established before held-out evaluation.

Model	CV AUC	Accuracy	Precision	Recall	F1	Test AUC
Logistic regression	0.911	0.803	0.767	0.821	0.793	0.932
Random forest	0.898	0.820	0.758	0.893	0.820	0.913

Logistic regression wins under the declared criterion. Random forest's higher test recall is reported rather than hidden; threshold choice should be tied to clinical costs in any future study.

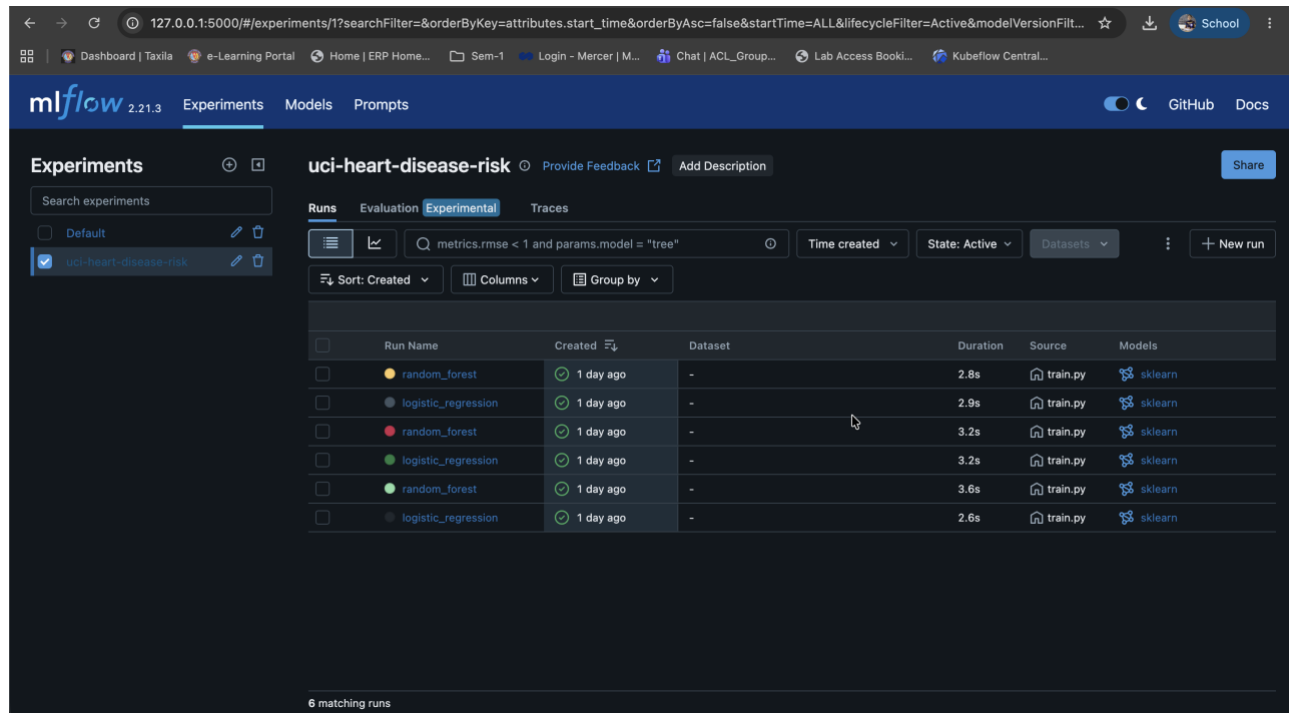


Held-out ROC curve for the selected pipeline.

3. Experiment tracking

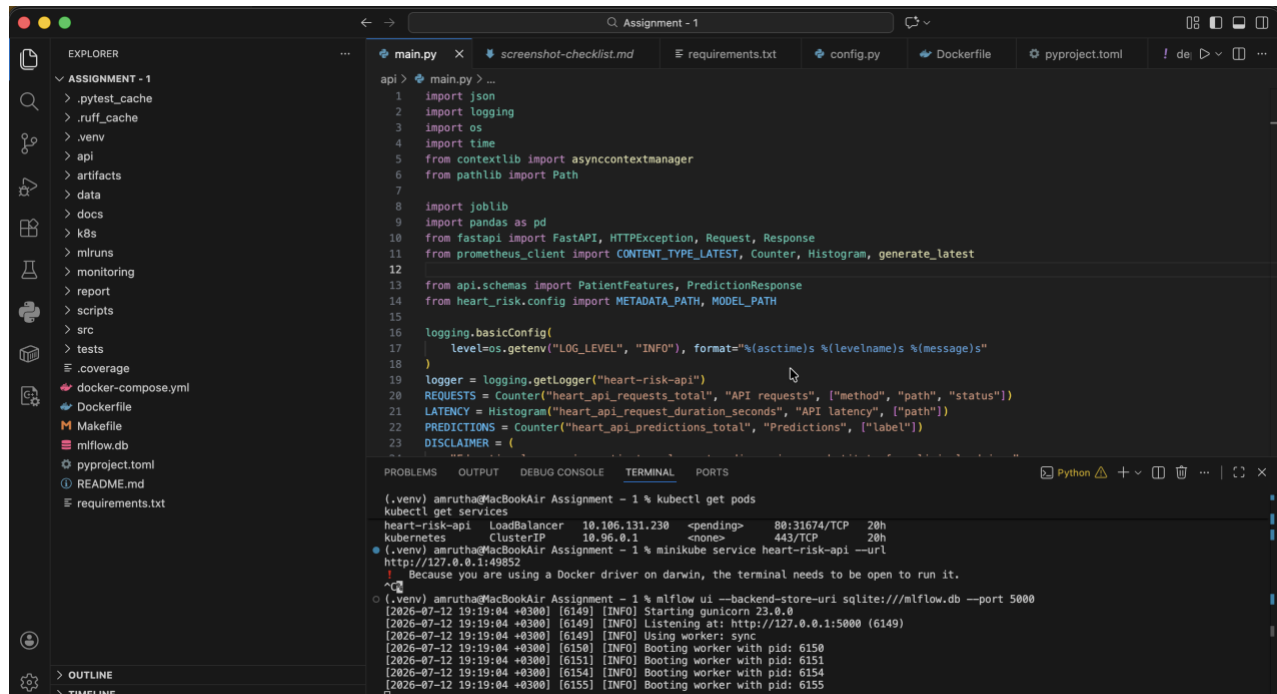
Each MLflow run stores hyperparameters, five-fold CV ROC-AUC, five test metrics, confusion matrix, ROC curve, an input example and the complete sklearn pipeline. SQLite supports a reproducible local demonstration, MLFLOW_TRACKING_URI pointing to a shared server.

Bash Command : `mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000`



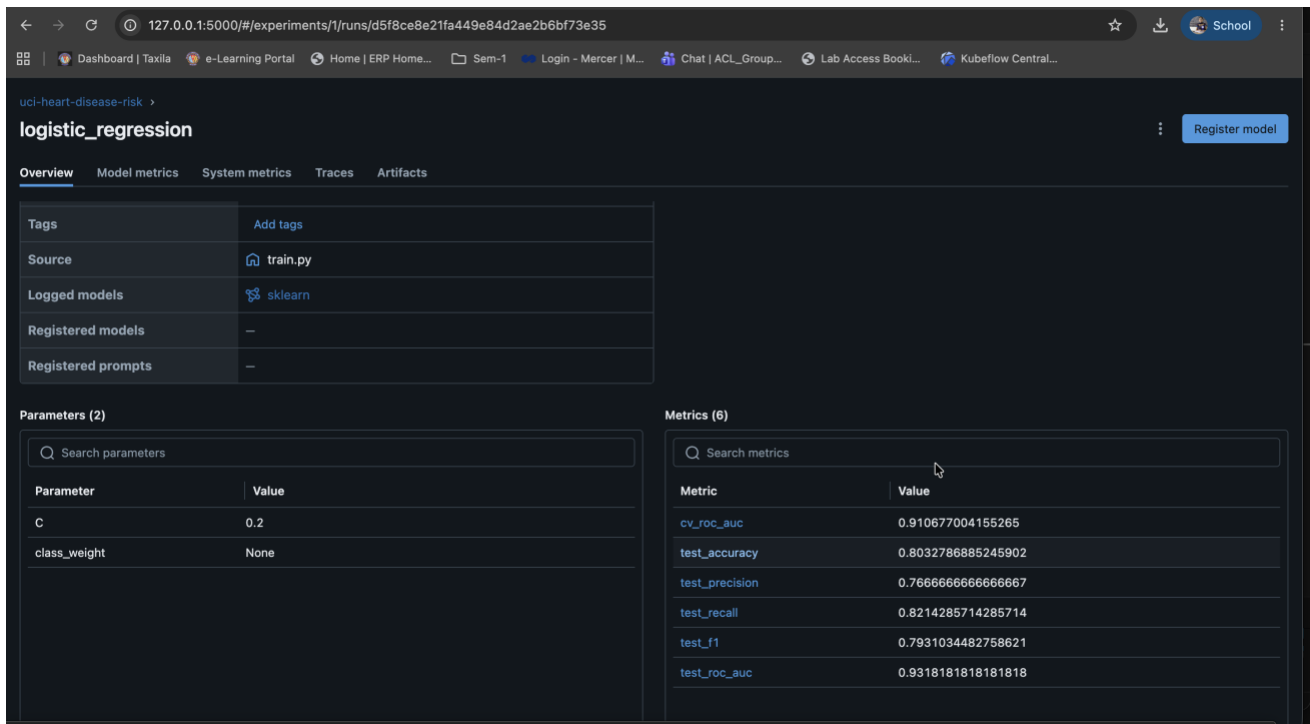
The screenshot shows the MLflow Experiments UI. The experiment name is 'uci-heart-disease-risk'. The 'Runs' tab is selected, showing a table of runs. The table has columns: Run Name, Created, Dataset, Duration, Source, and Models. There are 6 matching runs listed.

Run Name	Created	Dataset	Duration	Source	Models
random_forest	1 day ago	-	2.8s	train.py	sklearn
logistic_regression	1 day ago	-	2.9s	train.py	sklearn
random_forest	1 day ago	-	3.2s	train.py	sklearn
logistic_regression	1 day ago	-	3.2s	train.py	sklearn
random_forest	1 day ago	-	3.6s	train.py	sklearn
logistic_regression	1 day ago	-	2.6s	train.py	sklearn



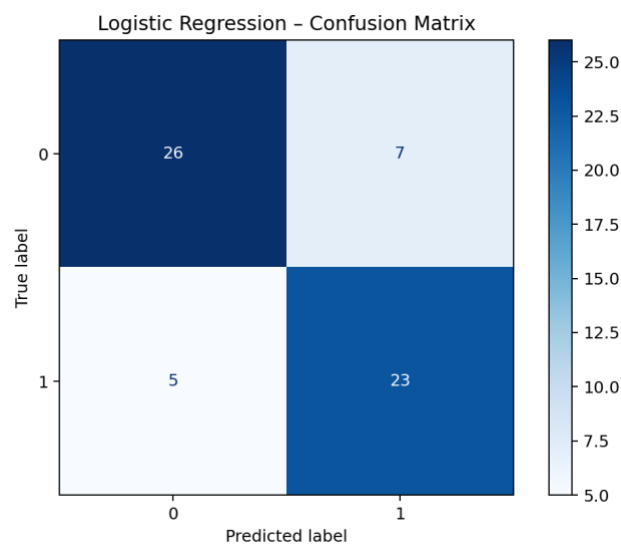
The screenshot shows a code editor with a file named 'main.py' and a terminal window. The terminal output shows the MLflow UI starting up and listening on port 5000.

```
api > main.py > ...
1 import json
2 import logging
3 import os
4 import time
5 from contextlib import asynccontextmanager
6 from pathlib import Path
7
8 import joblib
9 import pandas as pd
10 from fastapi import FastAPI, HTTPException, Request, Response
11 from prometheus_client import CONTENT_TYPE_LATEST, Counter, Histogram, generate_latest
12
13 from api.schemas import PatientFeatures, PredictionResponse
14 from heart_risk.config import METADATA_PATH, MODEL_PATH
15
16 logging.basicConfig(
17     level=os.getenv("LOG_LEVEL", "INFO"), format="%(asctime)s %(levelname)s %(message)s"
18 )
19 logger = logging.getLogger("heart-risk-api")
20 REQUESTS = Counter("heart_api_requests_total", "API requests", ["method", "path", "status"])
21 LATENCY = Histogram("heart_api_request_duration_seconds", "API latency", ["path"])
22 PREDICTIONS = Counter("heart_api_predictions_total", "Predictions", ["label"])
23 DISCLAIMER = (
24     "Because you are using a Docker driver on darwin, the terminal needs to be open to run it."
25 )
26
27 (.venv) amrutha@MacBookAir Assignment - 1 % kubect get pods
28 kubect get services
29 heart-risk-api LoadBalancer 10.106.131.230 <pending> 80:31674/TCP 20h
30 kubernetes ClusterIP 10.90.0.1 <none> 443/TCP 20h
31 (.venv) amrutha@MacBookAir Assignment - 1 % minikube service heart-risk-api --url
32 http://127.0.0.1:49852
33 ! Because you are using a Docker driver on darwin, the terminal needs to be open to run it.
34
35 (.venv) amrutha@MacBookAir Assignment - 1 % mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
36 [2026-07-12 19:19:04 +0300] [6149] [INFO] Starting unicorn 23.0.0
37 [2026-07-12 19:19:04 +0300] [6149] [INFO] Listening at: http://127.0.0.1:5000 (6149)
38 [2026-07-12 19:19:04 +0300] [6149] [INFO] Using worker: sync
39 [2026-07-12 19:19:04 +0300] [6150] [INFO] Booting worker with pid: 6150
40 [2026-07-12 19:19:04 +0300] [6151] [INFO] Booting worker with pid: 6151
41 [2026-07-12 19:19:04 +0300] [6154] [INFO] Booting worker with pid: 6154
42 [2026-07-12 19:19:04 +0300] [6155] [INFO] Booting worker with pid: 6155
```



4. Model Packaging and reproducibility

The selected Joblib artifact contains preprocessing and classification together, preventing training serving transformation skew. Metadata records the seed, timestamp, feature contract, sample counts, selection rule, metrics and limitations.



logistic_regression

Overview Model metrics System metrics Traces Artifacts

Tags	Add tags
Source	train.py
Logged models	sklearn
Registered models	—
Registered prompts	—

Parameters (2)

Parameter	Value
C	0.2
class_weight	None

Metrics (6)

Metric	Value
cv_roc_auc	0.910677004155265
test_accuracy	0.8032786885245902
test_precision	0.7666666666666667
test_recall	0.8214285714285714
test_f1	0.7931034482758621
test_roc_auc	0.9318181818181818

Selected-model confusion matrix on 61 held-out patients.

5. CI/CD Pipeline & Automated Testing

ruff check . for code quality checks.

pytest --cov=src/heart_risk --cov=api --cov-report=term-missing

```
[2026-07-12 19:46:57 +0300] [6149] [INFO] Shutting down: Master
(.venv) amrutha@MacBookAir Assignment - 1 % pytest --cov=src/heart_risk --cov=api --cov-report=term-missing
..... [100%]

----- coverage: platform darwin, python 3.11.4-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
api/__init__.py                      0      0   100%
api/main.py                          58      5    91%   30, 75-77, 93
api/schemas.py                     22      0   100%
src/heart_risk/__init__.py           1      0   100%
src/heart_risk/config.py             14      0   100%
src/heart_risk/data.py               25      6    76%   11-12, 27, 34-36
src/heart_risk/modeling.py           16      0   100%
TOTAL                                136     11    92%

6 passed, 1 warning in 1.25s
(.venv) amrutha@MacBookAir Assignment - 1 %
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[2026-07-12 19:46:57 +0300] [6149] [INFO] Shutting down: Master
(.venv) amrutha@MacBookAir Assignment - 1 % pytest --cov=src/heart_risk --cov=api --cov-report=term-missing
(.venv) amrutha@MacBookAir Assignment - 1 % ruff check .
src/heart_risk/config.py:1:1: I001 [*] Import block is un-sorted or un-formatted
1 | / from pathlib import Path
2 | | import os
3 | |
4 | | PROJECT_ROOT = Path(os.getenv("PROJECT_ROOT", Path.cwd())).resolve()
  | ~~~~~^ I001
= help: Organize imports

Found 1 error.
[*] 1 fixable with the '--fix' option.
(.venv) amrutha@MacBookAir Assignment - 1 % ruff check . --fix
Found 1 error (1 fixed, 0 remaining).
(.venv) amrutha@MacBookAir Assignment - 1 % ruff check .
All checks passed!
(.venv) amrutha@MacBookAir Assignment - 1 %
```

Docker file for deployment

```
EXPLORER
ASSIGNMENT - 1
  .pytest_cache
  .ruff_cache
  .venv
  api
  artifacts
  models
    heart_disease_pipeline.joblib
    model_comparison.csv
    model_metadata.json
  plots
  data
  docs
  k8s
  mlruns
  monitoring
  report
  scripts
  src
  tests
  .coverage
  docker-compose.yml
  Dockerfile
  Makefile
  mlflow.db
  pyproject.toml
  README.md
  requirements.txt

config.py Dockerfile X pyproject.toml deployment.yaml model_metadata.json Makefile eda_data ...

1 FROM python:3.11-slim AS runtime
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \
4     PYTHONUNBUFFERED=1 \
5     PIP_NO_CACHE_DIR=1 \
6     PROJECT_ROOT=/app
7
8 WORKDIR /app
9
10 COPY pyproject.toml ./
11 COPY src ./src
12 COPY api ./api
13 COPY artifacts/models ./artifacts/models
14
15 RUN python -m pip install --upgrade pip \
16     && python -m pip install . \
17     && test -f /app/artifacts/models/heart_disease_pipeline.joblib \
18     && test -f /app/artifacts/models/model_metadata.json
19
20 RUN groupadd --gid 10001 appgroup \
21     && useradd --create-home --uid 10001 --gid 10001 appuser \
22     && chown -R 10001:10001 /app
23
24 USER 10001:10001
25
26 EXPOSE 8000
27
28 HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
29     CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
30
31 CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]

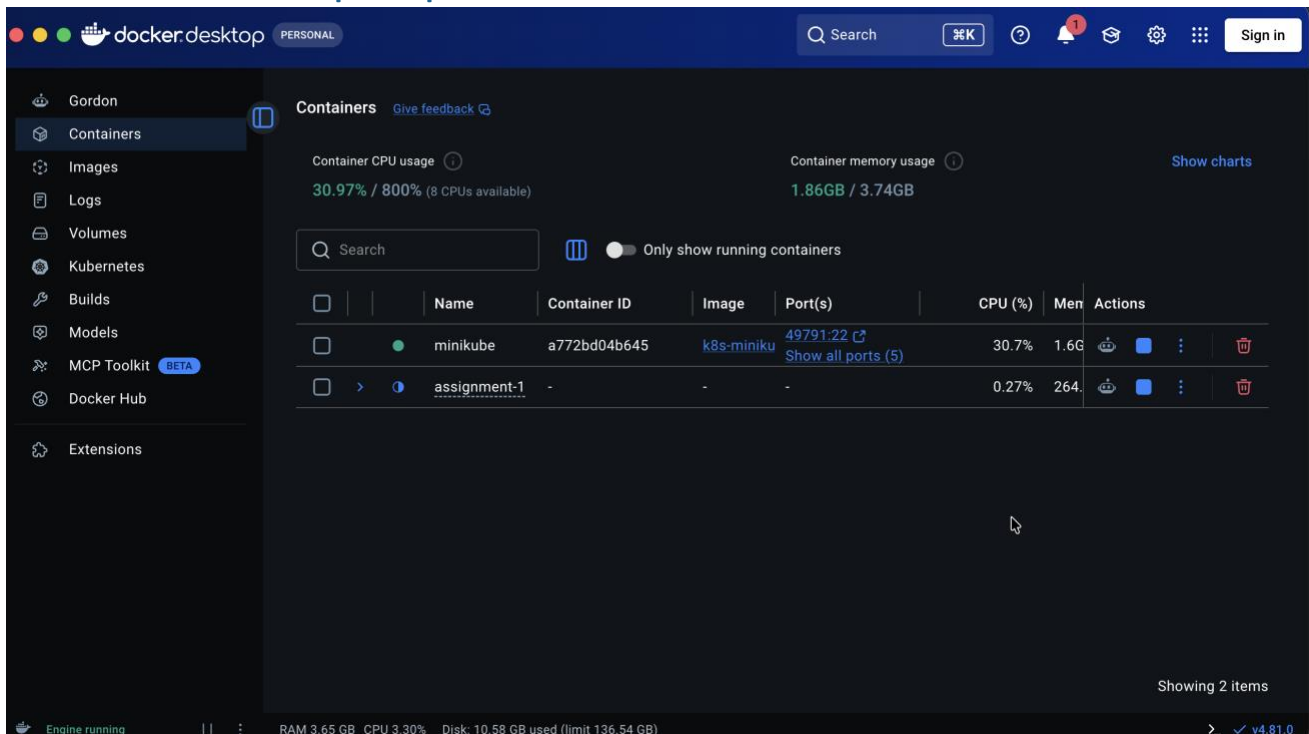
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) amrutha@MacBookAir Assignment - 1 % mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000
[2026-07-12 19:23:43 +0300] [6149] [INFO] Handling signal: winch
[2026-07-12 19:23:44 +0300] [6149] [INFO] Handling signal: winch
[2026-07-12 19:23:44 +0300] [6149] [INFO] Handling signal: winch
```

6. Model Containerization

The screenshot shows the Swagger UI for the 'Heart Disease Risk API' (version 1.0.0, OAS 3.1). The API is described as an 'ML Ops assignment API using the UCI Cleveland heart disease cohort'. The 'default' tab is selected, and the 'GET /health' endpoint is highlighted. The 'Parameters' section is empty. The 'Responses' section shows a 200 status code with a response body containing a JSON object: `{ "status": "healthy", "model": "logistic_regression" }`. The response headers are: `content-length: 58`, `content-type: application/json`, `date: Sat, 13 Jul 2024 15:21:28 GMT`, and `server: uvicorn`.

The screenshot shows the Swagger UI for the 'Heart Disease Risk API' (version 1.0.0, OAS 3.1). The API is described as an 'ML Ops assignment API using the UCI Cleveland heart disease cohort'. The 'default' tab is selected, and the 'POST /predict' endpoint is highlighted. The 'Parameters' section is empty. The 'Responses' section shows a 200 status code with a response body containing a JSON object: `{ "prediction": 0, "label": "absence", "probability": 0.25063, "model_version": "2826-87-11715-09:49.755199-00-00", "disclaimer": "Educational screening estimate only; not a diagnosis or substitute for clinical advice." }`. The response headers are: `content-length: 211`, `content-type: application/json`, `date: Sat, 13 Jul 2024 15:21:30 GMT`, and `server: uvicorn`.

Bash cmd : docker compose up --build



docker ps -a

```
(.venv) amrutha@MacBookAir Assignment - 1 % docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES	PORTS
a772bd04b645	gcr.io/k8s-minikube/kicbase:v0.0.50	"/usr/local/bin/entr..."	22 hours ago	Up 2 hours	minikube	127.0.0.1:49791->22/tcp
5d4d773979cd	grafana/grafana:11.5.2	"/run.sh"	23 hours ago	Exited (255) 2 hours ago	assignment-1-grafana-1	0.0.0.0:3000->3000/tcp
9978674e6e28	prom/prometheus:v3.2.1	"/bin/prometheus --c..."	23 hours ago	Exited (255) 2 hours ago	assignment-1-prometheus-1	0.0.0.0:9090->9090/tcp
4bfdbdbd5958	5bfd4302140a	"uvicorn api.main:ap..."	23 hours ago	Up 2 hours (healthy)	assignment-1-api-1	0.0.0.0:8000->8000/tcp

7. Production Deployment

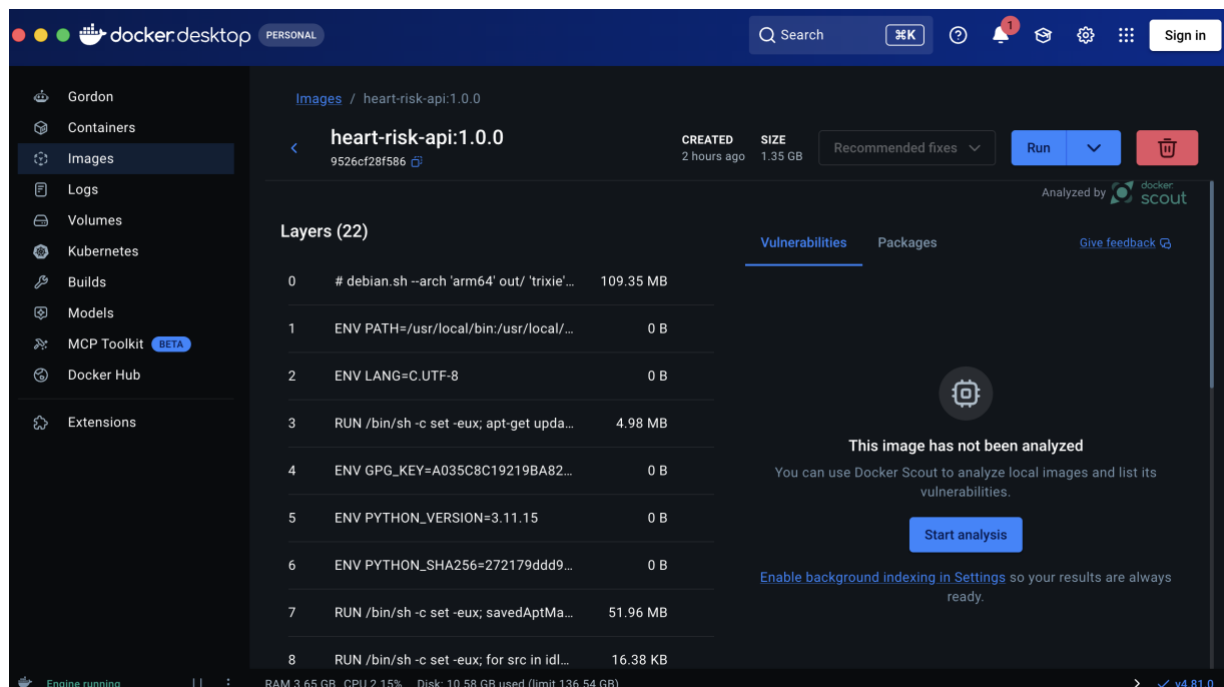
FastAPI exposes /health, /predict and /metrics. Pydantic validates all 13 fields, rejects unexpected keys and applies defensible range checks. Responses contain prediction, label, probability, model timestamp. Request logs omit supplied medical values.

Quality gates

- Data schema, target conversion, missingness and duplicate handling.
- Both candidate pipelines fit and return two-class probabilities.
- Health and prediction endpoints succeed with a controlled test artifact.
- Out-of-range patient inputs return HTTP 422.
- Verified result: 6 passed, 92% coverage; Ruff clean.

Container design

The Python 3.11 slim image runs as non-root UID 10001, contains the trained model, exposes port 8000 and includes a health check. The build context excludes development data, tests, notebooks and MLflow state. Docker Compose adds Prometheus and Grafana for a one-command local demonstration.



Github actions or devops creates Python 3.11 environment, runs Ruff and Pytest, retrieves and validates the dataset, trains both candidates, uploads model/evaluation artifacts and builds the Docker image. Shell failures stop the job and the workflow uses read-only repository permission.

Kubernetes deploys two replicas behind a LoadBalancer Service. Readiness and liveness probes call /health. Explicit resource requests and limits control scheduling. Security settings require non-root execution, prohibit privilege escalation, drop Linux capabilities and mount a read-only root filesystem.

Local deployment sequence

- Starting Minikube and pointing the Docker client at its image.
- Build heart-risk-api:1.0.0, then apply the k8s directory.
- Wait for deployment rollout; confirm two Ready pods.
- Use minikube service heart-risk-api --url and call /predict.

BASH:

```
docker compose up --build
```

```
minikube start
```

```
docker build -t heart-risk-api:1.0.0 .
```

```
kubectl apply -f k8s/
```

```
kubectl rollout status deployment/heart-risk-api
```

```
kubectl get pods
```

```
kubectl get services
```

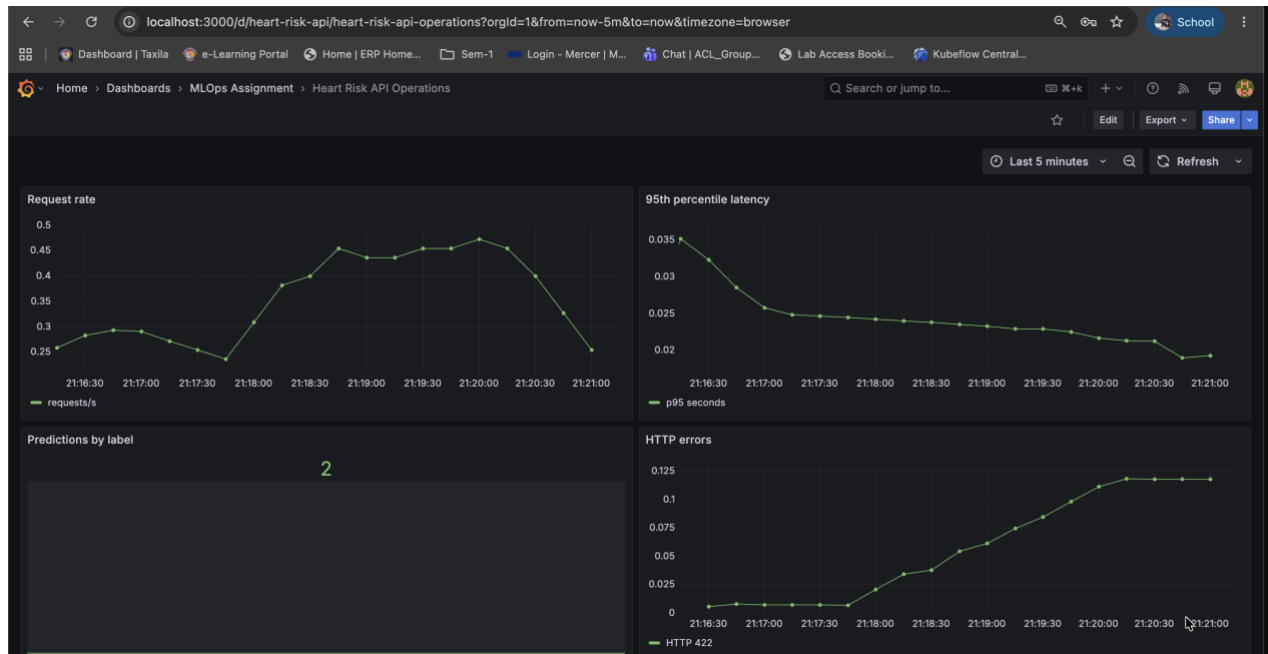
```
Assignment - 1

ResourceManager"
grafana-1 | logger=app-registry t=2026-07-11T18:14:07.704592007Z level=info msg="app registry initialized"
grafana-1 | logger=plugins.update.checker t=2026-07-11T18:14:07.861698298Z level=info msg="Update check succeeded" duration=552.276292ms
grafana-1 | logger=plugin.angular detectorsprovider.dynamic t=2026-07-11T18:14:08.023138049Z level=info msg="Patterns update finished" durat
ion=711.99875ms
grafana-1 | logger=grafana.update.checker t=2026-07-11T18:14:08.023725049Z level=info msg="Update check succeeded" duration=714.396167ms
api-1 | 2026-07-11 18:14:08,084 INFO request method=GET path=/health status=200
api-1 | INFO: 127.0.0.1:39484 - "GET /health HTTP/1.1" 200 OK
api-1 | 2026-07-11 18:14:09,113 INFO request method=GET path=/metrics status=200
api-1 | INFO: 172.18.0.3:38160 - "GET /metrics HTTP/1.1" 200 OK
grafana-1 | logger=plugin.installer t=2026-07-11T18:14:09.423757008Z level=info msg="Installing plugin" pluginId=grafana-lokiexplore-app ve
rsion=
grafana-1 | logger=installer.fs t=2026-07-11T18:14:09.518797841Z level=info msg="Downloaded and extracted grafana-lokiexplore-app v1.0.10 z
ip successfully to /var/lib/grafana/plugins/grafana-lokiexplore-app"
grafana-1 | logger=plugins.registration t=2026-07-11T18:14:09.542699883Z level=info msg="Plugin registered" pluginId=grafana-lokiexplore-ap
p
grafana-1 | logger=plugin.backgroundinstaller t=2026-07-11T18:14:09.542727341Z level=info msg="Plugin successfully installed" pluginId=graf
ana-lokiexplore-app version= duration=2.230905709s
grafana-1 | logger=plugin.backgroundinstaller t=2026-07-11T18:14:09.542806674Z level=info msg="Installing plugin" pluginId=grafana-pyroscope
e-app version=
grafana-1 | logger=plugin.installer t=2026-07-11T18:14:10.253253008Z level=info msg="Installing plugin" pluginId=grafana-pyroscope-app vers
ion=
grafana-1 | logger=installer.fs t=2026-07-11T18:14:10.32662755Z level=info msg="Downloaded and extracted grafana-pyroscope-app v1.17.0 zip
successfully to /var/lib/grafana/plugins/grafana-pyroscope-app"
grafana-1 | logger=plugins.registration t=2026-07-11T18:14:10.338921675Z level=info msg="Plugin registered" pluginId=grafana-pyroscope-app
grafana-1 | logger=plugin.backgroundinstaller t=2026-07-11T18:14:10.3389453Z level=info msg="Plugin successfully installed" pluginId=grafan
a-pyroscope-app version= duration=796.128542ms
api-1 | 2026-07-11 18:14:14,130 INFO request method=GET path=/metrics status=200
api-1 | INFO: 172.18.0.3:38160 - "GET /metrics HTTP/1.1" 200 OK
api-1 | 2026-07-11 18:14:19,103 INFO request method=GET path=/metrics status=200
api-1 | INFO: 172.18.0.3:38160 - "GET /metrics HTTP/1.1" 200 OK
api-1 | 2026-07-11 18:14:24,122 INFO request method=GET path=/metrics status=200
api-1 | INFO: 172.18.0.3:38160 - "GET /metrics HTTP/1.1" 200 OK
api-1 | INFO: 172.18.0.3:38160 - "GET /metrics HTTP/1.1" 200 OK
api-1 | 2026-07-11 18:14:29,108 INFO request method=GET path=/metrics status=200
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh

(.venv) amrutha@MacBookAir Assignment - 1 % kubectl apply -f k8s/
deployment.apps/heart-risk-api configured
service/heart-risk-api unchanged
(.venv) amrutha@MacBookAir Assignment - 1 % kubectl rollout status deployment/heart-risk-api
deployment "heart-risk-api" successfully rolled out
(.venv) amrutha@MacBookAir Assignment - 1 % kubectl get pods
kubectl get services
NAME READY STATUS RESTARTS AGE
heart-risk-api-7bf77474df-pkng5 1/1 Running 0 33s
heart-risk-api-7bf77474df-qzqml 1/1 Running 0 45s
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
heart-risk-api LoadBalancer 10.106.131.230 <pending> 80:31674/TCP 20h
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 20h
```

8. Monitoring, logging and production response



Prometheus scrapes request counters, prediction counters and a latency histogram every five seconds. The provisioned Grafana dashboard displays request rate, p95 latency, outcomes by label and HTTP errors. Application logs record timestamp, method, route and status without patient feature payloads.

9. Reproduction, conclusion and references

Clean reproduction

- Create a virtual environment and install requirements.txt.
- Run download_data.py, run_eda.py and train.py.
- Run pytest with coverage and ruff check.
- Start MLflow on port 5000 and FastAPI on port 8000.
- Build Docker/Compose; deploy k8s manifests with Minikube.

API or local access: <http://127.0.0.1:8000/docs>

Demonstration video:

The screenshot shows a web browser window with the address bar displaying `0.0.0.0:8000/docs#/default/predict_predict_post`. The browser's top navigation bar includes links to 'Dashboard | Taxila', 'e-Learning Portal', 'Home | ERP Home...', 'Sem-1', 'Login - Mercer | M...', 'Chat | ACL_Group...', 'Lab Access Booki...', and 'Kubeflow Central...'. The main content area is titled 'Edit Value | Schema' and contains a JSON object representing a patient's medical data:

```
{
  "age": 55,
  "sex": 1,
  "cp": 1,
  "trestbps": 70,
  "chol": 81,
  "fbs": 1,
  "restecg": 2,
  "thalach": 51,
  "exang": 1,
  "oldpeak": -3,
  "slope": 5,
  "ca": 3,
  "thal": 5
}
```

Below the JSON editor, there are two buttons: 'Execute' (highlighted in blue) and 'Clear'. Under the 'Responses' section, there is a 'Curl' tab showing the equivalent curl command:

```
curl -X 'POST' \
  'http://0.0.0.0:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "age": 55,
    "sex": 1,
    "cp": 1,
    "trestbps": 70,
    "chol": 81,
    "fbs": 1
```